

36 EJERCICIOS PRÁCTICOS DEL MÓDULO 8

Divididos en básicos, intermedios, avanzados y un mini–proyecto.

EJERCICIOS BÁSICOS (1–12)

1. Crear un script que muestre tu nombre

Mostrar:

echo "Mi nombre es ..."

2. Script que muestre la fecha actual

date

3. Script que muestre el directorio actual

pwd

4. Script que reciba un argumento y lo muestre

Ejemplo:

./script.sh hola

5. Script que cuente del 1 al 10 (for)

6. Script que pida al usuario su nombre y lo salude

7. Script que muestre el contenido de /etc

8. Script que muestre los 5 procesos que más CPU consumen

Usar ps aux.

9. Script que muestre espacio libre del disco

df -h

10. Script que cree una carpeta automáticamente

11. Script que cree un archivo con tu nombre y la fecha actual

12. Script que revise si un archivo existe o no

Usar if [-f archivo].

Todos los ejercicios en un solo Script

```
nvim ejercicios_B
File Edit View Search Terminal Help
1 #!/bin/bash$
2 echo "-- Iniciando Ejercicios Básicos --"$
3 echo "Presiona Enter para continuar entre ejercicios..."$
4 read -r$
5 # 4. Recibir y mostrar argumento$
6 if [ $# -eq 0 ]; then$
7     ARG="Nadie" # Por defecto$
8 else$
9     ARG="$1"$
10 fi$
11 $
12 # 1. Mostrar nombre$
13 echo "=== EJERCICIO 1 ==="$
14 echo "Mi nombre es $ARG" # Personalizado [memory:44]$
15 read -r$
16 $
17 # 2. Fecha actual$
18 echo "=== EJERCICIO 2 - date ==="$
19 date$
20 read -r$
21 $
22 # 3. Directorio actual$
23 echo "=== EJERCICIO 3 - pwd ==="$
24 pwd$
25 read -r$
26 # 5. Contar del 1 al 10 (for)$
27 echo "=== EJERCICIO 5 - 1 al 10 ==="$
28 for i in {1..10}; do$
29     echo "Número: $i"$
30 done$
31 read -r$
32 # 6. Pedir nombre y saludar$
33 echo "=== EJERCICIO 6 - Nombre ==="$
34 echo "Ingresa tu nombre:"$
35 read -r NOMBRE$
36 echo "¡Hola, $NOMBRE! Bienvenido a Bash."$
37 read -r$
38 # 7. Contenido de /etc (solo primeros 10 para no saturar)$
39 echo "=== EJERCICIO 7 listado /etc==="$
40 echo "Primeros 10 archivos en /etc:"$
41 ls /etc | head -10$
42 read -r$
43 $
```

```
nvim ejercicios_Basicos.sh
File Edit View Search Terminal Help
43 $
44 # 8. 5 procesos con más CPU$
45 echo "=== EJERCICIO 8 CPU ==="$
46 echo "Top 5 procesos por CPU:"$
47 ps aux --sort=-%cpu | head -6 # Incluye header$
48 read -r$
49 $
50 # 9. Espacio libre del disco$
51 echo "=== EJERCICIO 9 Espacio Libre ==="$
52 df -h | head -6 # Solo primeros 6 para tabla limpia$
53 read -r$
54 $
55 # 10. Crear carpeta automáticamente$
56 echo "=== EJERCICIO 10 Carpeta ==="$
57 CARPETA="mi_carpeta_automatica_$(date +%d%m%Y)"$
58 mkdir -p "$CARPETA"$
59 echo "✓ Carpeta creada: $CARPETA"$
60 ls -ld "$CARPETA"$
61 read -r$
62 $
63 # 11. Crear archivo con nombre y fecha$
64 echo "=== EJERCICIO 11 Archivo ==="$
65 ARCHIVO="$CARPETA/JCSantos_$(date +%d%m%Y_%H%M).txt"$
66 echo "Creado Flumperio - $(date)" > "$ARCHIVO"$
67 echo "✓ Archivo creado: $ARCHIVO"$
68 cat "$ARCHIVO"$
69 read -r$
70 $
71 # 12. Revisar si archivo existe$
72 echo "=== EJERCICIO 12 Existe File ?==="$
73 if [ -f "$ARCHIVO" ]; then$
74     echo "✓ El archivo $ARCHIVO EXISTE."$
75     echo "Tamaño: $(wc -c < "$ARCHIVO") bytes"$
76 else$
77     echo "✗ El archivo $ARCHIVO NO existe."$
78 fi$
79 $
80 ARCHIVO="/home/jcsantos/no_existe.txt"$
81 if [ -f "$ARCHIVO" ]; then$
82     echo "✓ El archivo $ARCHIVO EXISTE."$
83     echo "Tamaño: $(wc -c < "$ARCHIVO") bytes"$
84 else$
85     echo "✗ El archivo $ARCHIVO NO existe."$
86 fi$
87 $
```

✓ EJERCICIOS INTERMEDIOS (13–24)

13. Script que lea dos números y los sume

```
# 1. Suma de dos input
echo "=== EJERCICIO 1 ==="
echo "Introduce el numero 1:" # Personalizado [memory:44]
read -r NUMERO1
echo "Introduce el numero 2:"
read -r NUMERO2
SUMA=$((NUMERO1+NUMERO2))
echo "La suma del numero $NUMERO1 + $NUMERO2 es: $SUMA"
```

14. Script que compruebe si un servicio está activo

systemctl is-active ssh

```
# 2. Monitorizacion del Servicio
echo "=== EJERCICIO 2 ==="
echo "Introduce el nombre del servicio a monitorizar:"
read -r SERVICIO
systemctl is-active "$SERVICIO"
read -r
```

15. Script que reinicie automáticamente un servicio si está caído

```
# 3. Reiniciar el servicio
echo "=== EJERCICIO 3 ==="
ESTADO=$(systemctl is-active "$SERVICIO")
if [[ "$ESTADO" == "active" ]]; then
    echo "$SERVICIO activo"
    systemctl restart "$SERVICIO"
else
    echo "$SERVICIO inactivo: $ESTADO"
    systemctl start "$SERVICIO"
fi
read -r
```

16. Script que haga ping a un servidor y avise si falla

```
# 5. Ping aviso si falla
echo "=== EJERCICIO 4 ==="
echo "Dime la direccion para comprobar"
read -r SERVICIO
if ping -c 1 $SERVICIO &>/dev/null; then
    echo "Online"
else
    echo "Desconectado"
fi
read -r
```

17. Script que liste todos los usuarios del sistema

`cut -d: -f1 /etc/passwd`

```
# 5. Scrip lista usuarios
echo "=== EJERCICIO 5 ==="
cut -d: -f1 /etc/passwd | more
read -r
```

18. Script que genere un log en /var/log/local/

19. Script que busque archivos mayores de 100MB

20. Script que monitorice RAM cada 10 segundos

21. Script que cree 10 archivos con nombres secuenciales

22. Script que elimine archivos temporales automáticamente

23. Script que comprima una carpeta en .tar.gz

24. Script que verifique conexión a Internet

Hacer ping a 8.8.8.8.

 EJERCICIOS AVANZADOS (25–36)

25. Script que haga copia de seguridad de /home

26. Script que compruebe si un puerto está abierto

`ss -tulnp | grep 22`

27. Script que envíe un mensaje de alerta si un disco supera el 80%

28. Script con menú interactivo (1. Info – 2. Red – 3. Salir)

29. Script que muestre los 10 archivos más grandes del sistema

30. Script que genere informes diarios de estado

CPU, RAM, disco, red.

31. Script que haga un backup incremental

32. Script que detecte procesos zombie

ps aux | grep Z

33. Script que reinicie automáticamente un servicio si falla

34. Script que registre la IP pública

curl ifconfig.me

35. Script que revise logs del sistema y filtre “error”

36. Programar tu script con cron para que se ejecute cada día

MINI-PROYECTO DEL MÓDULO 8 – Sistema de Monitorización Completo

Objetivo:

Crear un sistema automático de monitorización realista en Ubuntu.

Debe incluir un script llamado monitor.sh que:

✓ Muestre:

- Fecha y hora
- Uso de CPU

- Uso de RAM
- 5 procesos más pesados
- Espacio libre en disco
- Estado de un servicio (apache2 o ssh)
- Conectividad a Internet

✓ Guarde un log en:

/var/log/monitor.log

✓ Cree alertas si:

- CPU > 80%
- RAM > 80%
- Disco > 80%

✓ Se ejecute automáticamente con cron cada 15 minutos:

`*/15 * * * * /ruta/monitor.sh`

✓ Incluya documentación técnica del proyecto:

- Código usado
- Explicación
- Capturas
- Solución a errores